

José Gabriel de Almeida Vieira (Reuso da Software e Metodologias Ágeis)
Pedro Henrique Balbino Rocha (Reuso da Software e Metodologias Ágeis)
Raniel Ferreira Athayde (Reuso da Software e Metodologias Ágeis)

CodeWeaver Framework: relatório técnico sobre reuso de software em arquitetura de microserviços

Repositório: <<https://github.com/Raniel-Athayde/CodeWeaver>>

José Gabriel de Almeida Vieira (Reuso da Software e Metodologias Ágeis)
Pedro Henrique Balbino Rocha (Reuso da Software e Metodologias Ágeis)
Raniel Ferreira Athayde (Reuso da Software e Metodologias Ágeis)

CodeWeaver Framework: relatório técnico sobre reuso de software em arquitetura de microserviços

Repositório: <<https://github.com/Raniel-Athayde/CodeWeaver>>

Relatório apresentado ao Instituto de Computação da Universidade Federal de Alagoas como atividade acadêmica da disciplina Reuso de Software e Metodologias Ágeis, período 2026.1, ministrada pelo professor Arturo Hernandez Dominguez.

Universidade Federal de Alagoas
Instituto de Computação

Maceió
2026

RESUMO

Este relatório apresenta o CodeWeaver Framework, um projeto didático desenvolvido em Python para demonstrar conceitos de reuso de software, arquitetura orientada a microserviços e construção de compiladores e interpretadores extensíveis. O framework organiza o processamento de código em um pipeline fixo composto pelas etapas de tokenização, análise sintática, otimização, execução e notificação. Ao mesmo tempo, permite que partes variáveis do comportamento sejam substituídas por implementações específicas, como diferentes analisadores léxicos, analisadores sintáticos, interpretadores e serviços externos. A implementação evidencia a presença dos conceitos de *Frozen Spots* e *Hotspots*, a aplicação do padrão de projeto *Template Method* e o uso de injeção de dependências para reduzir acoplamento entre os componentes. Também são apresentados diagramas de classes, casos de uso, componentes e fluxo de execução, além de pontos fortes, limitações e possibilidades de evolução do sistema.

Palavras-chave: framework; microserviços; reuso de software; padrões de projeto; Python.

SUMÁRIO

1	INTRODUÇÃO	4
1.1	Objetivos	5
1.2	Mapeamento arquitetural inicial	5
2	METODOLOGIA	7
2.1	Desenvolvimento orientado a microserviços	7
2.2	Componentes do framework	8
2.2.1	CodeWeaverEngine	8
2.2.2	Interfaces abstratas	9
2.2.3	Gateway	10
2.2.4	Analyzer	11
2.2.5	Notifier	11
2.2.6	Serviço base	11
2.3	Modelagem do sistema	12
2.3.1	Diagrama de classes	12
2.3.2	Diagrama de casos de uso	13
2.3.3	Diagrama de componentes	14
2.4	Padrão de Projeto Utilizado	15
3	ANÁLISE E DISCUSSÃO	17
3.1	Frozen Spots e Hotspots	17
3.2	Arquitetura dos microserviços	17
3.3	Implementação das interfaces	18
4	CONCLUSÃO	19
	REFERÊNCIAS BIBLIOGRÁFICAS	21

1 INTRODUÇÃO

O desenvolvimento de compiladores e interpretadores é uma atividade central na área de linguagens de programação, pois permite transformar programas escritos em uma linguagem de alto nível em representações compreensíveis por máquinas, ambientes de execução ou outros sistemas computacionais. Tradicionalmente, esse processo envolve etapas como análise léxica, análise sintática, geração de uma representação intermediária, otimização e execução ou tradução do código. Embora essas etapas sejam recorrentes em diferentes linguagens, a construção de um compilador completo costuma exigir grande esforço técnico, principalmente quando cada projeto precisa implementar do zero estruturas como *lexers*, *parsers*, árvores de sintaxe abstrata e mecanismos de interpretação.

Nesse contexto, frameworks surgem como alternativa para organizar e reutilizar partes comuns desse processo. Em engenharia de software, um framework pode ser entendido como uma estrutura reutilizável que define um fluxo geral de funcionamento, mas permite que determinados pontos sejam customizados conforme a aplicação desejada. Essa ideia é especialmente relevante no domínio de compiladores, pois muitas linguagens compartilham uma arquitetura semelhante de processamento, ainda que possuam regras sintáticas, semânticas e formas de execução diferentes.

Trabalhos recentes reforçam que infraestruturas reutilizáveis reduzem a complexidade na criação de compiladores e linguagens específicas de domínio. O MLIR exemplifica essa abordagem ao oferecer uma base comum para diferentes domínios e arquiteturas, inclusive em compiladores especializados para TPU (LATTNER et al., 2020; HU et al., 2022). De forma complementar, *language workbenches* e ferramentas do ecossistema Spoolfax mostram como ambientes especializados auxiliam a especificação, evolução e manipulação de linguagens (WOOD, 2014; KARSAI et al., 2014; MOSSES, 2023).

Frameworks voltados para compiladores e interpretadores normalmente funcionam por meio da separação entre um núcleo fixo e pontos variáveis de customização. O núcleo fixo define o fluxo principal de processamento, por exemplo: receber o código-fonte, tokenizar a entrada, construir uma árvore de sintaxe abstrata, aplicar otimizações e executar ou traduzir o resultado. Esse fluxo pode ser reaproveitado por diferentes linguagens. Por outro lado, os pontos variáveis permitem que o desenvolvedor implemente regras específicas, como a gramática da linguagem, a forma de interpretação dos comandos ou as estratégias de otimização.

Essa separação está relacionada aos conceitos de *Frozen Spots* e *Hotspots*. Os *Frozen Spots* representam as partes fixas do framework, isto é, os elementos que permanecem inalterados independentemente da linguagem implementada. Já os *Hotspots* representam os pontos de extensão, nos quais o usuário do framework pode inserir comportamentos específicos. Em um framework para compiladores e interpretadores, o pipeline de execução

pode ser considerado um *Frozen Spot*, enquanto o *lexer*, o *parser* e o interpretador podem ser vistos como *Hotspots*.

Com base nesse contexto, este relatório apresenta o CodeWeaver Framework, um framework modular para compiladores e interpretadores desenvolvido com o objetivo de demonstrar, de forma prática, a aplicação desses conceitos. A proposta do CodeWeaver é fornecer uma engine central responsável por controlar o fluxo fixo de processamento de código, enquanto componentes específicos, como o analisador léxico, o analisador sintático e o interpretador, podem ser substituídos conforme a linguagem desejada.

1.1 OBJETIVOS

O objetivo geral deste trabalho é desenvolver e apresentar um framework para compiladores e interpretadores, denominado CodeWeaver Framework, fundamentado em reuso de software, extensibilidade e arquitetura orientada a microserviços. Como objetivos específicos, destacam-se:

- projetar uma estrutura reutilizável para o processamento de código-fonte;
- implementar uma engine central responsável pelo pipeline de tokenização, análise sintática, otimização, execução e notificação;
- definir pontos fixos e pontos de extensão por meio dos conceitos de *Frozen Spots* e *Hotspots*;
- aplicar o padrão *Template Method* no funcionamento da classe `CodeWeaverEngine`;
- integrar os microserviços Gateway, Analyzer e Notifier por meio de comunicação HTTP;
- representar a modelagem do sistema por meio de diagramas UML e arquiteturais.

1.2 MAPEAMENTO ARQUITETURAL INICIAL

A Tabela 1 resume a relação entre os conceitos arquiteturais de frameworks e sua aplicação no CodeWeaver.

Além disso, o CodeWeaver adota uma arquitetura inspirada em microserviços, composta por um Gateway principal, um serviço Analyzer e um serviço Notifier. O Gateway atua como ponto de entrada da aplicação e coordena a execução do framework. O Analyzer representa um serviço externo de otimização da árvore de sintaxe abstrata, enquanto o Notifier simula o envio de notificações após a execução do código. Essa organização demonstra que os pontos de extensão de um framework não precisam estar limitados ao código local, podendo também ser distribuídos em serviços independentes.

Tabela 1 – Mapeamento entre conceitos de frameworks e o CodeWeaver

Conceito arquitetural	Definição geral em frameworks	Mapeamento no CodeWeaver Framework
<i>Frozen Spots</i>	Componentes fixos e estruturais que ditam o fluxo principal da aplicação.	Engine central, pipeline de execução e coordenação de fluxo pelo Gateway.
<i>Hotspots</i>	Pontos flexíveis de extensão customizáveis pelo desenvolvedor.	Analizador léxico, analisador sintático, interpretador e microserviços distribuídos Analyzer e Notifier.

Fonte: Elaborado pelo autor (2026).

2 METODOLOGIA

A metodologia utilizada combinou pesquisa conceitual, implementação dos módulos Python e documentação da arquitetura do framework. O desenvolvimento considerou a responsabilidade de cada componente, a ordem do pipeline de execução, os pontos fixos e variáveis da aplicação e a forma como a comunicação HTTP é usada para integrar serviços externos.

O código-fonte do CodeWeaver Framework foi desenvolvido pelos autores deste relatório e está disponível em repositório público hospedado no GitHub de Raniel Ferreira Athayde, permitindo a consulta da implementação, dos serviços e dos módulos descritos neste relatório (VIEIRA; ROCHA; ATHAYDE, 2026).

2.1 DESENVOLVIMENTO ORIENTADO A MICROSERVIÇOS

O CodeWeaver Framework foi desenvolvido seguindo uma abordagem orientada a microserviços, na qual a aplicação é dividida em componentes independentes, cada um responsável por uma parte específica do processamento. Em vez de concentrar toda a lógica em um único sistema monolítico, o projeto separa suas responsabilidades em três serviços principais: Gateway, Analyzer e Notifier.

O Gateway representa o serviço central da aplicação. Ele recebe as requisições do usuário, instancia a engine do framework e coordena o fluxo de execução do código. O Analyzer atua como um serviço externo de análise e otimização, recebendo uma Árvore de Sintaxe Abstrata (AST) e aplicando possíveis modificações antes da etapa de interpretação. O Notifier, por sua vez, é responsável por registrar ou simular notificações após a execução do código.

Essa organização permite demonstrar um dos principais benefícios da arquitetura de microserviços: a separação de responsabilidades. Cada serviço possui uma função bem definida e pode ser modificado, substituído ou expandido sem exigir alterações profundas nos demais componentes. Além disso, a comunicação entre os serviços ocorre por meio de requisições HTTP, o que torna a integração mais flexível e próxima de arquiteturas distribuídas utilizadas em aplicações reais. A separação em camadas e responsabilidades também dialoga com princípios clássicos de arquitetura de aplicações empresariais discutidos por Fowler (2002).

No projeto, a comunicação é realizada principalmente a partir da classe **CodeWeaver-Engine**, que pode chamar serviços externos configurados por URL. O serviço Analyzer é acessado por meio do endpoint `</optimize>`, enquanto o serviço Notifier é acessado por meio do endpoint `</notify>`. Dessa forma, a engine mantém seu fluxo principal, mas pode delegar partes específicas do processamento para componentes externos.

Os principais conceitos responsáveis pela aplicação são:

- **Microserviços:** separação da aplicação em serviços independentes, como Gateway, Analyzer e Notifier;
- **Reuso de software:** criação de uma estrutura comum que pode ser reaproveitada para diferentes linguagens;
- **Frozen Spots:** partes fixas do framework, como o fluxo principal de execução definido pela engine;
- **Hotspots:** pontos de customização, como *lexer*, *parser*, *interpreter*, Analyzer e Notifier;
- **Injeção de dependências:** a engine recebe objetos externos responsáveis por tokenizar, analisar e interpretar o código;
- **Comunicação HTTP:** integração entre os serviços por meio de requisições POST;
- **Template Method:** definição de um algoritmo fixo que delega etapas específicas para componentes variáveis.

Assim, a metodologia de desenvolvimento do CodeWeaver combina conceitos de engenharia de software, frameworks reutilizáveis, padrões de projeto e arquitetura distribuída, resultando em uma aplicação didática para construção de compiladores e interpretadores extensíveis.

2.2 COMPONENTES DO FRAMEWORK

O CodeWeaver Framework é composto por módulos que representam tanto o núcleo reutilizável quanto os serviços responsáveis pela execução distribuída da aplicação. Os principais componentes são `CodeWeaverEngine`, as interfaces abstratas, o Gateway, o Analyzer, o Notifier e o serviço base.

2.2.1 CODEWEAVERENGINE

A classe `CodeWeaverEngine`, localizada no arquivo `<framework/engine.py>`, é o núcleo do framework. Ela concentra o fluxo principal de processamento de código, funcionando como o ponto fixo da arquitetura. Seu construtor recebe implementações de *lexer*, *parser* e *interpreter*, além de URLs opcionais para os serviços Analyzer e Notifier.

Código 2.1 – Construtor da classe `CodeWeaverEngine`

```
1 class CodeWeaverEngine:
```

```
2     def __init__(self, lexer, parser, interpreter, analyzer_url=None, notifier_url=
      None):
3         self.lexer = lexer
4         self.parser = parser
5         self.interpreter = interpreter
6         self.analyzer_url = analyzer_url
7         self.notifier_url = notifier_url
```

Esse trecho mostra que a engine não cria diretamente os componentes de análise léxica, sintática e interpretação. Em vez disso, esses componentes são recebidos externamente. Essa característica torna o framework flexível, pois diferentes linguagens podem fornecer diferentes implementações para essas etapas.

O principal método da engine é `compile_and_run`, responsável por executar o pipeline fixo do framework.

Código 2.2 – Pipeline principal de compilação e execução

```
1 def compile_and_run(self, source_code):
2     tokens = self._tokenize(source_code)
3     ast = self._parse(tokens)
4     ast = self._optimize(ast)
5     output = self._execute(ast)
6     self._notify(output, execution_time)
7     return result
```

Esse método representa o fluxo de compilação/interpretação. Primeiro, o código-fonte é transformado em tokens. Em seguida, os tokens são convertidos em uma AST. Depois, a AST pode ser otimizada por um serviço externo. Por fim, a AST é executada pelo interpretador, e o resultado é enviado ao serviço de notificação.

2.2.2 INTERFACES ABSTRATAS

O arquivo `<framework/interfaces.py>` define os contratos que devem ser seguidos pelas implementações concretas de linguagem. Essas interfaces são importantes porque padronizam a forma como os componentes customizáveis interagem com a engine.

Código 2.3 – Interfaces abstratas do framework

```
1 class BaseLexer(ABC):
2     @abstractmethod
3     def tokenize(self, code: str) -> list:
4         pass
5
6 class BaseParser(ABC):
7     @abstractmethod
8     def parse(self, tokens: list) -> dict:
9         pass
10
```

```
11 class BaseInterpreter(ABC):
12     @abstractmethod
13     def execute(self, ast: dict) -> str:
14         pass
```

Essas classes definem três responsabilidades centrais em um compilador ou interpretador. **BaseLexer** define a operação de tokenização, **BaseParser** define a construção da representação sintática e **BaseInterpreter** define a execução do programa. Ao usar classes abstratas, o framework garante que qualquer linguagem implementada respeite a estrutura mínima necessária para funcionar com a engine.

2.2.3 GATEWAY

O Gateway é o serviço principal da aplicação e está localizado em `<gateway/app.py>`. Ele configura a engine e expõe uma API HTTP para receber o código a ser processado.

Código 2.4 – Configuração da engine no Gateway

```
1 engine = CodeWeaverEngine(
2     lexer=PlaceholderLexer(),
3     parser=PlaceholderParser(),
4     interpreter=PlaceholderInterpreter(),
5     analyzer_url=os.getenv("ANALYZER_URL", "http://localhost:5001"),
6     notifier_url=os.getenv("NOTIFIER_URL", "http://localhost:5002")
7 )
```

Nesse trecho, o Gateway injeta implementações concretas na engine. No estado atual do projeto, são usadas classes *placeholder*, ou seja, implementações simples criadas para demonstrar a estrutura do framework. Mesmo assim, essa organização permite que futuramente essas classes sejam substituídas por uma linguagem real, sem necessidade de modificar a engine.

O Gateway também define o endpoint `</api/compile>`, responsável por receber o código enviado pelo usuário.

Código 2.5 – Endpoint principal de compilação

```
1 if self.path == '/api/compile':
2     data = self.get_post_data()
3     code = data.get('code', '')
4     result = engine.compile_and_run(code)
5     self.send_json(result)
```

Esse fluxo mostra que o Gateway atua como ponto de entrada. Ele recebe o código, chama o método `compile_and_run` da engine e devolve o resultado em formato JSON.

2.2.4 ANALYZER

O Analyzer é um microserviço localizado em <analyzer/app.py>. Sua função é receber a AST gerada pelo parser e aplicar uma otimização simples.

Código 2.6 – Otimização executada pelo Analyzer

```
1 if self.path == '/optimize':
2     ast = self.get_post_data()
3
4     if ast.get('type') == 'PrintStatement':
5         ast.setdefault('metadata', {})
6         ast['metadata']['optimized'] = True
7         ast['metadata']['service_info'] = "Otimizado via Framework Service"
8
9     self.send_json(ast)
```

Esse serviço representa um *hotspot* externo. Ele pode modificar a AST antes que ela seja interpretada. Atualmente, a otimização é simples e ocorre apenas quando a AST possui o tipo `PrintStatement`. No entanto, a estrutura permite que otimizações mais complexas sejam implementadas futuramente.

2.2.5 NOTIFIER

O Notifier é o microserviço responsável por registrar uma notificação ao final da execução. Ele está localizado em <notifier/app.py>.

Código 2.7 – Notificação enviada ao fim do processamento

```
1 if self.path == '/notify':
2     data = self.get_post_data()
3     print(f"[NOTIFICADOR]: {data.get('message')}")
4     self.send_json({"status": "ok"})
```

Esse componente recebe uma mensagem enviada pela engine e imprime uma notificação no terminal. Embora seja uma implementação simples, ela demonstra como o framework pode acionar serviços externos após a execução do código.

2.2.6 SERVIÇO BASE

O arquivo <framework/service.py> define uma estrutura comum para os serviços HTTP do projeto.

Código 2.8 – Serviço base para comunicação HTTP

```
1 class BaseServiceHandler(BaseHTTPRequestHandler):
2     def get_post_data(self):
3         content_length = int(self.headers['Content-Length'])
4         return json.loads(self.rfile.read(content_length))
```

```
5
6     def send_json(self, data, status=200):
7         self.send_response(status)
8         self.send_header('Content-type', 'application/json')
9         self.end_headers()
10        self.wfile.write(json.dumps(data).encode('utf-8'))
```

Esse componente evita repetição de código, pois fornece métodos comuns para leitura de dados JSON e envio de respostas JSON. Assim, Gateway, Analyzer e Notifier podem reutilizar a mesma base de comunicação.

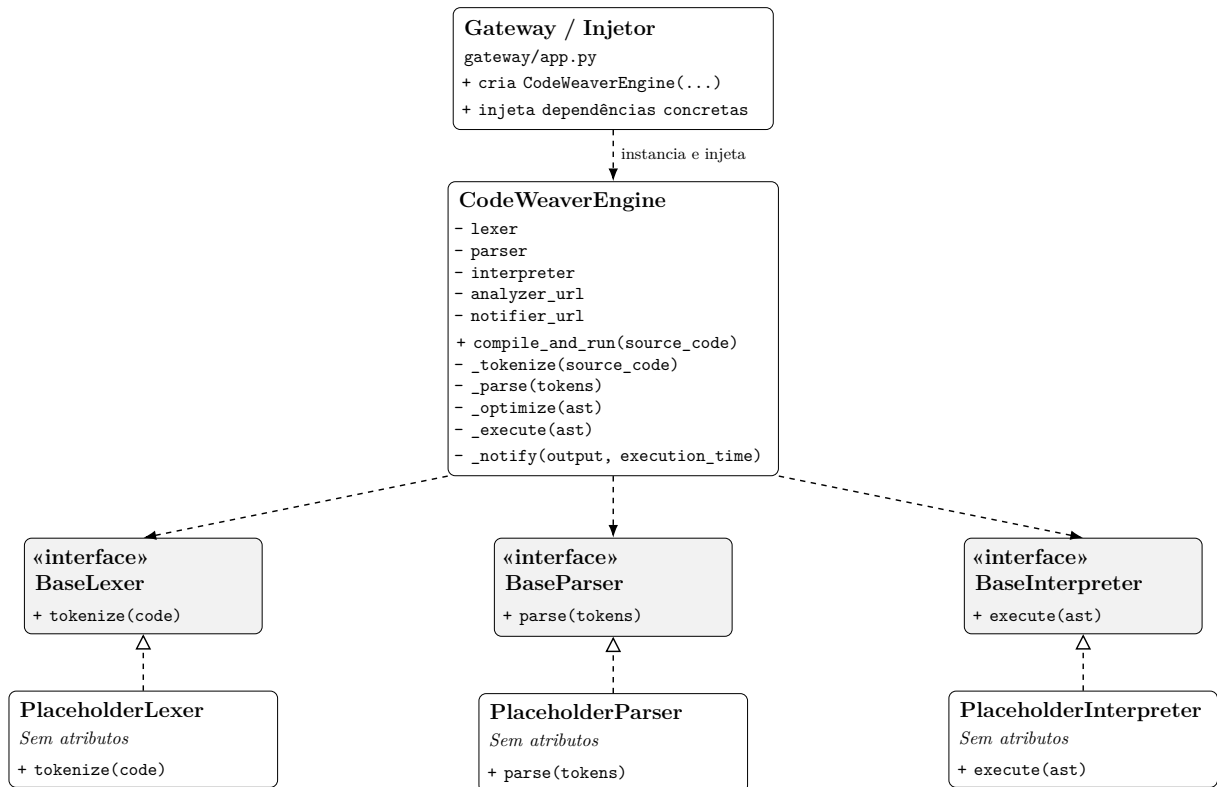
2.3 MODELAGEM DO SISTEMA

A modelagem do CodeWeaver Framework é representada por diferentes diagramas UML, destacando aspectos estruturais e comportamentais do sistema. As figuras foram elaboradas em TikZ, permitindo que os diagramas sejam renderizados diretamente no PDF final.

2.3.1 DIAGRAMA DE CLASSES

A Figura 1 apresenta a relação entre a `CodeWeaverEngine` e as interfaces abstratas usadas pela engine.

Figura 1 – Diagrama de classes do CodeWeaver Framework



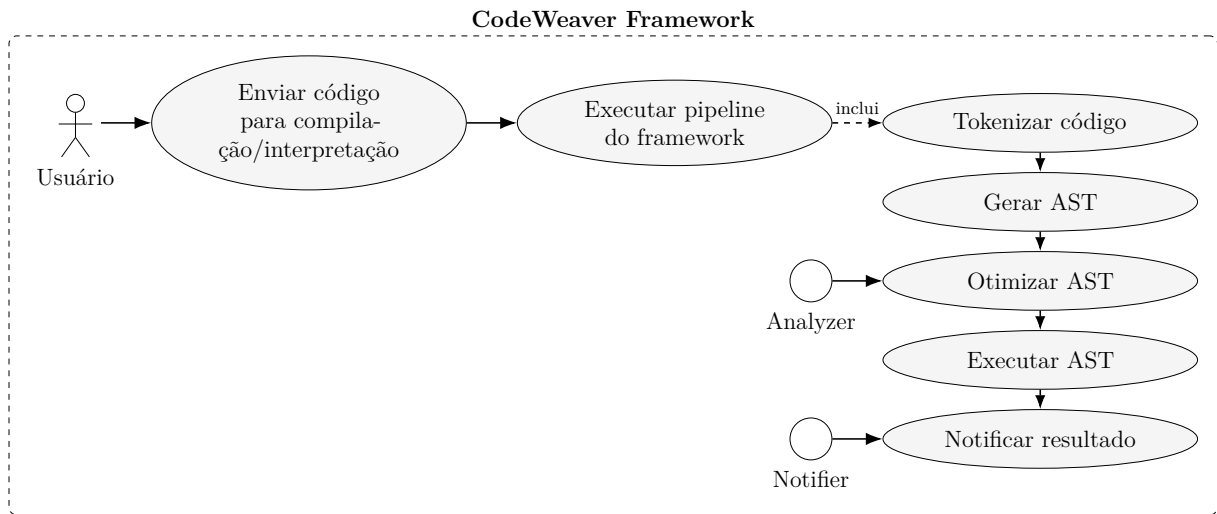
Fonte: Elaborado pelo autor (2026).

Nesse diagrama, a **CodeWeaverEngine** depende das interfaces, e não diretamente das implementações concretas. As classes **PlaceholderLexer**, **PlaceholderParser** e **PlaceholderInterpreter** representam implementações reais usadas no Gateway e não possuem atributos ou variáveis de instância, pois apenas implementam os métodos exigidos pelas interfaces. As linhas tracejadas com triângulo vazado indicam a realização dos contratos definidos pelas interfaces. O Gateway/Injetor representa o módulo responsável por instanciar a engine e injetar essas dependências.

2.3.2 DIAGRAMA DE CASOS DE USO

A Figura 2 evidencia que o principal caso de uso é o envio de código para processamento. A partir disso, o framework executa suas etapas internas e interage com os serviços externos quando necessário.

Figura 2 – Diagrama de casos de uso do CodeWeaver Framework



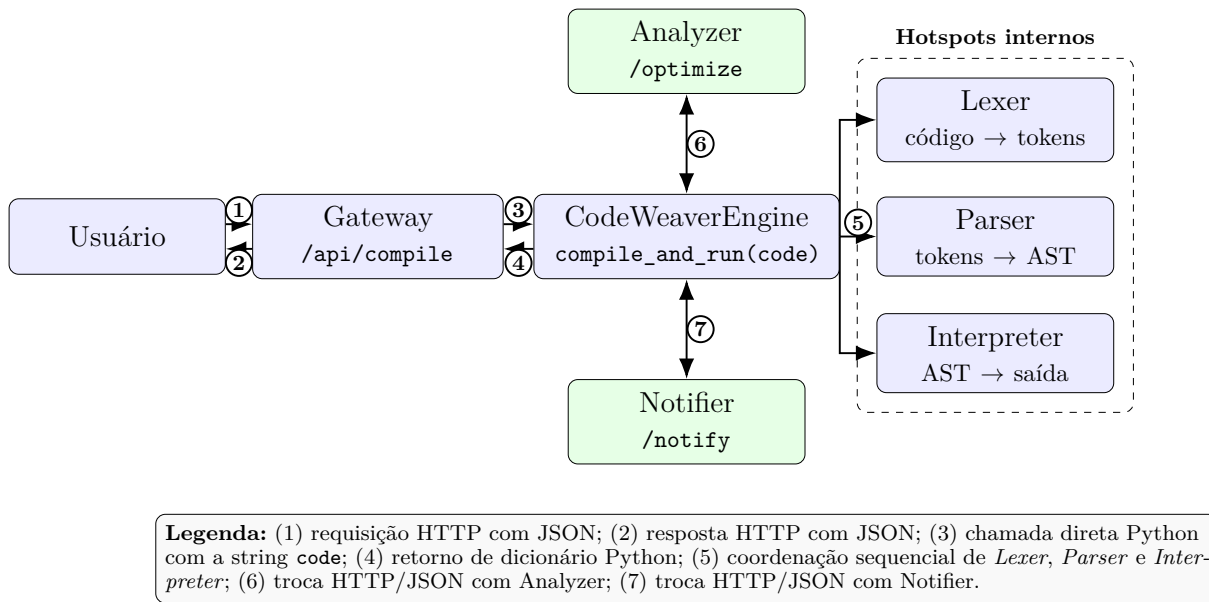
Fonte: Elaborado pelo autor (2026).

2.3.3 DIAGRAMA DE COMPONENTES

A Figura 3 apresenta a organização dos principais componentes do CodeWeaver Framework e explicita a separação entre a camada HTTP, o núcleo de processamento e os pontos de extensão. O Usuário não acessa diretamente a **CodeWeaverEngine**; ele envia uma requisição HTTP ao Gateway, normalmente com um corpo em JSON contendo o campo `code`. O Gateway interpreta essa requisição, extrai a string de código-fonte e aciona a engine por meio de uma chamada direta de método dentro do mesmo processo Python.

Dessa forma, a comunicação entre o Gateway e a **CodeWeaverEngine** não ocorre por HTTP nem por JSON. O Gateway chama o método `compile_and_run(code)` passando apenas a string com o código recebido. Após executar o pipeline, a engine retorna um dicionário Python estruturado, contendo informações como status, saída produzida e tempo de execução. Somente depois disso o Gateway serializa esse dicionário em JSON e o envia ao Usuário como corpo de uma resposta HTTP.

Figura 3 – Diagrama de componentes e comunicação entre serviços



Fonte: Elaborado pelo autor (2026).

No interior da engine, os componentes *Lexer*, *Parser* e *Interpreter* também não se comunicam diretamente entre si. Eles funcionam como *Hotspots* independentes, coordenados pela **CodeWeaverEngine**. A engine chama o *Lexer* para transformar o código-fonte em tokens, repassa esses tokens ao *Parser* para gerar a AST e, após a etapa de otimização, entrega a AST ao *Interpreter* para produzir a saída da execução. Assim, o resultado de cada etapa é recebido pela engine e utilizado como entrada para o próximo estágio do pipeline.

Os serviços Analyzer e Notifier representam extensões externas ao núcleo da engine. Diferentemente da relação entre Gateway e engine, a comunicação da **CodeWeaverEngine** com esses serviços ocorre por requisições HTTP com dados em JSON. O Analyzer recebe a AST para otimização e devolve uma versão otimizada, enquanto o Notifier recebe informações sobre o resultado da execução e retorna um status de notificação. Com isso, o diagrama evidencia três tipos distintos de interação: HTTP com JSON entre Usuário e Gateway, chamada direta Python entre Gateway e engine, e HTTP com JSON entre a engine e os microserviços externos.

2.4 PADRÃO DE PROJETO UTILIZADO

O principal padrão de projeto utilizado no CodeWeaver Framework é o *Template Method*. Segundo Gamma et al. (1994), esse padrão define a estrutura geral de um algoritmo em uma operação principal, deixando algumas etapas para serem implementadas

ou customizadas por subclasses, métodos auxiliares ou objetos externos.

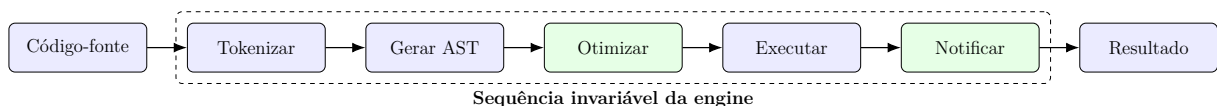
No contexto do CodeWeaver, o método `compile_and_run` da classe `CodeWeaverEngine` funciona como o *Template Method*. Ele define a ordem fixa do processamento: tokenizar, analisar, otimizar, executar e notificar. Essa sequência não muda conforme a linguagem utilizada. O que pode mudar são as implementações concretas de cada etapa.

Esse padrão é adequado para frameworks porque permite separar o que é fixo do que é variável. O framework controla o fluxo principal, enquanto o desenvolvedor fornece os comportamentos específicos. No CodeWeaver, essa separação aparece da seguinte forma:

- a sequência do pipeline é fixa e pertence à engine;
- a tokenização é delegada ao *lexer*;
- a análise sintática é delegada ao *parser*;
- a execução é delegada ao *interpreter*;
- a otimização pode ser delegada ao Analyzer;
- a notificação pode ser delegada ao Notifier.

A Figura 4 representa o fluxo fixo definido pelo *Template Method*.

Figura 4 – Pipeline fixo definido pelo Template Method



Fonte: Elaborado pelo autor (2026).

Portanto, o *Template Method* contribui diretamente para a proposta do framework, pois garante que toda linguagem implementada siga o mesmo processo geral de compilação/interpretação, mas com liberdade para personalizar os detalhes de cada etapa. Essa abordagem aumenta o reuso, melhora a organização do código e facilita a criação de novas linguagens a partir da mesma estrutura base. Além disso, o projeto utiliza conceitos próximos de injeção de dependências, pois a engine recebe externamente os objetos *lexer*, *parser* e *interpreter*. Isso evita que a engine dependa de uma linguagem específica e torna o framework mais flexível e extensível.

3 ANÁLISE E DISCUSSÃO

3.1 FROZEN SPOTS E HOTSPOTS

Um dos principais objetivos do projeto é demonstrar a diferença entre *Frozen Spots* e *Hotspots*. Os *Frozen Spots* são partes fixas do framework. Eles representam decisões arquiteturais que não devem mudar a cada uso do sistema. No CodeWeaver, o principal *Frozen Spot* é o método `compile_and_run` da classe `CodeWeaverEngine`. Esse método define a sequência obrigatória do processamento:

1. tokenizar o código-fonte;
2. gerar a AST;
3. otimizar a AST;
4. executar a AST;
5. notificar o resultado.

Essa ordem permanece a mesma independentemente da linguagem ou da implementação concreta utilizada. Os *Hotspots*, por sua vez, são pontos de customização. Eles permitem que o usuário do framework adapte comportamentos sem modificar o núcleo. No CodeWeaver, os principais *Hotspots* são o *lexer*, o *parser*, o *interpreter*, o `analyzer_url` e o `notifier_url`. Essa separação torna o framework mais reutilizável, pois o mesmo pipeline pode ser usado com diferentes linguagens ou estratégias de processamento.

3.2 ARQUITETURA DOS MICROSSERVIÇOS

A arquitetura de microserviços do projeto separa responsabilidades em componentes independentes. O Gateway atua como ponto de entrada e coordena o uso do framework. O Analyzer realiza uma otimização simples sobre a AST, marcando determinados nós com metadados. O Notifier recebe informações sobre a execução e imprime uma mensagem de notificação.

Essa divisão tem vantagens importantes:

- **Modularidade:** cada serviço possui uma responsabilidade específica;
- **Extensibilidade:** o Analyzer e o Notifier podem ser substituídos por versões mais completas;

- **Independência:** serviços externos podem ser ativados, desativados ou modificados sem alterar o núcleo da engine;
- **Demonstração didática:** o projeto mostra como um framework pode integrar extensões locais e distribuídas.

Por outro lado, a arquitetura também traz desafios. A comunicação HTTP exige tratamento de falhas, configuração de portas e execução simultânea de três processos. O próprio framework já trata algumas falhas de forma defensiva, por exemplo quando o Analyzer está indisponível e a engine prossegue com a AST original.

3.3 IMPLEMENTAÇÃO DAS INTERFACES

O arquivo `<framework/interfaces.py>` define três classes abstratas: `BaseLexer`, `BaseParser` e `BaseInterpreter`. Essas classes funcionam como contratos para implementações futuras. Qualquer linguagem que deseje usar o framework deve fornecer classes concretas que implementem os métodos `tokenize`, `parse` e `execute`.

No Gateway atual, existem implementações *placeholder*, isto é, implementações simples apenas para demonstrar o funcionamento do pipeline. O `PlaceholderLexer` retorna o código recebido em uma lista, o `PlaceholderParser` cria uma estrutura com os tokens e o `PlaceholderInterpreter` retorna uma mensagem com os dados processados. Embora simples, essa estrutura já demonstra a intenção do framework: separar o algoritmo fixo da lógica customizável.

4 CONCLUSÃO

Este relatório apresentou o desenvolvimento e a documentação do CodeWeaver Framework, um framework voltado para compiladores e interpretadores, construído com o objetivo de demonstrar conceitos importantes de engenharia de software, reuso de código e arquitetura orientada a microserviços. A proposta principal do framework foi organizar o processo de compilação e interpretação em uma estrutura reutilizável, separando o fluxo fixo de execução dos pontos de customização que podem variar conforme a linguagem implementada.

Na construção do CodeWeaver, foi adotada uma abordagem baseada em *Frozen Spots* e *Hotspots*. O *Frozen Spot* principal está na classe `CodeWeaverEngine`, responsável por definir a sequência fixa do processamento: tokenização, análise sintática, otimização, execução e notificação. Essa estrutura representa o núcleo do framework e garante que qualquer linguagem construída sobre ele siga um mesmo fluxo geral. Já os *Hotspots* aparecem nos componentes customizáveis, como o *lexer*, o *parser*, o *interpreter* e os serviços externos Analyzer e Notifier, permitindo que diferentes comportamentos sejam implementados sem alterar a base central do sistema.

Também foi aplicado o padrão de projeto *Template Method*, utilizado para definir o esqueleto do algoritmo de compilação/interpretação. Esse padrão se mostrou adequado para a proposta do projeto, pois permite manter a ordem das etapas sob controle da engine, enquanto delega partes específicas para componentes especializados. Dessa forma, o framework favorece o reuso, reduz o acoplamento entre módulos e facilita a extensão para novas linguagens ou estratégias de processamento.

Outro ponto relevante foi a adoção de uma arquitetura orientada a microserviços. A divisão entre Gateway, Analyzer e Notifier demonstra como responsabilidades distintas podem ser separadas em serviços independentes. O Gateway atua como ponto de entrada da aplicação, o Analyzer representa um serviço de otimização da AST e o Notifier simula a comunicação de resultados após a execução. Essa organização torna o sistema mais modular e permite que futuras melhorias sejam realizadas em serviços específicos sem comprometer toda a aplicação.

Apesar de seu caráter didático, o CodeWeaver apresenta uma base conceitual importante. Entre seus principais pontos fortes estão a clareza na separação entre núcleo fixo e pontos de extensão, o uso objetivo do padrão *Template Method*, o baixo acoplamento entre a engine e as implementações concretas, a possibilidade de integrar serviços externos por URL e a simplicidade do código para fins acadêmicos. Esses aspectos mostram, de maneira prática, como um framework para compiladores e interpretadores pode ser estruturado para reaproveitar partes comuns do processo de linguagem, ao mesmo tempo em que permite a personalização das etapas específicas.

Entretanto, o projeto ainda possui limitações. As implementações atuais de *lexer*, *parser* e *interpreter* são simplificadas e funcionam como exemplos iniciais. Além disso, não há uma linguagem real completamente implementada no código principal atual, a comunicação entre serviços é simples e não possui autenticação, o tratamento de rotas HTTP é básico e a execução exige iniciar manualmente três serviços em terminais separados. Essas limitações não invalidam o projeto, mas indicam que ele ainda está mais próximo de uma prova de conceito acadêmica do que de uma solução pronta para produção.

Como possibilidades de melhoria, o CodeWeaver poderia evoluir com a implementação de uma linguagem de exemplo completa, como uma MathLang funcional, a criação de um arquivo `docker-compose.yml` para subir os três serviços com um único comando, a melhoria da interface web, a documentação de exemplos de entrada e saída, a inclusão de logs estruturados, o tratamento de erros mais detalhado, a configuração centralizada por arquivo `.env` e a criação de documentação específica sobre como desenvolver novos *Hotspots*. Essas melhorias aumentariam a robustez do projeto e facilitariam seu uso por outros desenvolvedores.

Conclui-se, portanto, que o CodeWeaver Framework cumpre sua proposta de demonstrar a construção de um framework extensível para compiladores e interpretadores. Por meio da combinação entre *Template Method*, *Frozen Spots*, *Hotspots* e microserviços, o projeto evidencia como princípios de arquitetura de software podem ser aplicados para criar sistemas mais modulares, reutilizáveis e fáceis de evoluir. Assim, o trabalho contribui para compreender, de forma prática, como frameworks desse tipo podem apoiar o desenvolvimento de linguagens e ferramentas de interpretação.

REFERÊNCIAS BIBLIOGRÁFICAS

FOWLER, M. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.

GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1994.

HU, P. et al. *TPU-MLIR: A Compiler for TPU Using MLIR*. 2022. Disponível em: <<https://arxiv.org/abs/2210.15016>>. Acesso em: 6 jun. 2026.

KARSAI, G. et al. *Design Guidelines for Domain Specific Languages*. 2014. Disponível em: <<https://arxiv.org/abs/1409.2378>>. Acesso em: 6 jun. 2026.

LATTNER, C. et al. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. 2020. Disponível em: <<https://arxiv.org/abs/2002.11054>>. Acesso em: 6 jun. 2026.

MOSSES, P. D. *Using Spoofax to Support Online Code Navigation*. 2023. Disponível em: <<https://arxiv.org/abs/2303.03015>>. Acesso em: 6 jun. 2026.

VIEIRA, J. G. d. A.; ROCHA, P. H. B.; ATHAYDE, R. F. *CodeWeaver Framework*. 2026. Repositório de código-fonte hospedado no GitHub de Raniel Ferreira Athayde. Disponível em: <<https://github.com/Raniel-Athayde/CodeWeaver.git>>. Acesso em: 6 jun. 2026.

WOOD, G. *Martta: A C++ Language Workbench*. 2014. Disponível em: <<https://arxiv.org/abs/1403.3752>>. Acesso em: 6 jun. 2026.